

Software Reverse Engineering Course

- Site: OnHexGroup.ir
- Youtube: @onhexgroup
- Telegram: onhex_ir
- Twitter: @onhexgroup
- Github: onhexgroup

Windows
(x86 / x64)

By: OnhexGroup

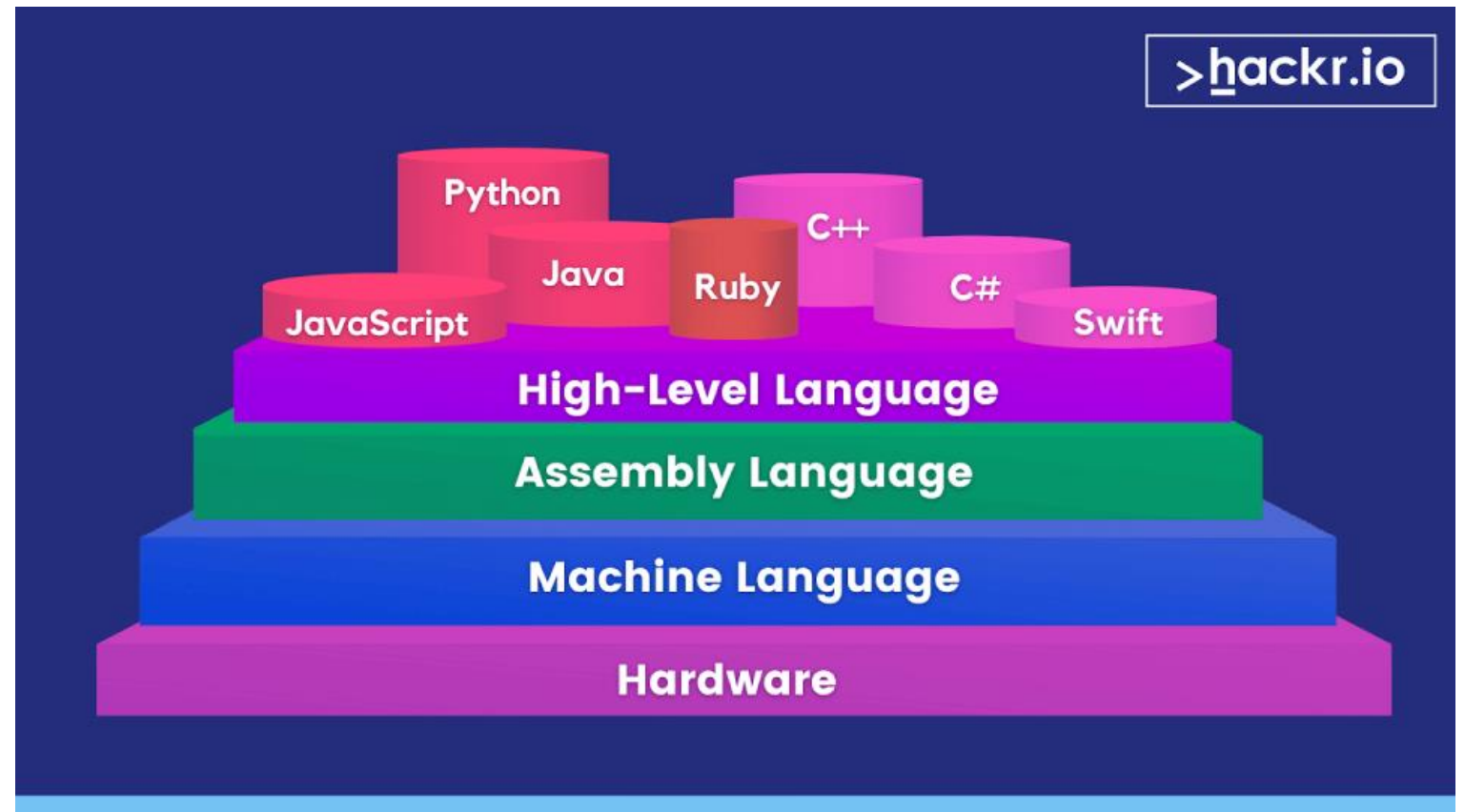
Onhexgroup.ir

Types of Programming Languages

- **Low-Level Languages**
 - First Generation: Machine Language
 - Second Generation: Assembly Language
- **High-Level Languages**
 - Examples: C, C#, Python, Go, etc.
- **Middle-Level Languages**
 - Defined based on memory management capabilities
 - Example: C
- **The concept of low-level and high-level is relative.**

Youtube: Onhexgroup

Types of Programming Languages



Onhexgroup.ir

Types of High-Level Programming Languages

- **Interpreted Programming Languages**
- **Compiled Programming Languages**
- **Hybrid (Interpreted & Compiled) Programming Languages**

Telegram: onhex_ir

Interpreted Programming Languages

■ **Examples:**

- Python, Ruby, JavaScript, Lua, PowerShell, etc.

■ **Characteristics:**

- Translated by an **Interpreter**
- Instructions are read line by line and immediately converted to machine code
- Line-by-line interpretation occurs on every execution (slower performance)
- Requires the interpreter to be installed on the target system
- Source code is visible
- Easy development, maintenance, and debugging

Youtube: Onhexgroup

Compiled Programming Languages

■ **Examples:**

- C, C#, Go, etc.

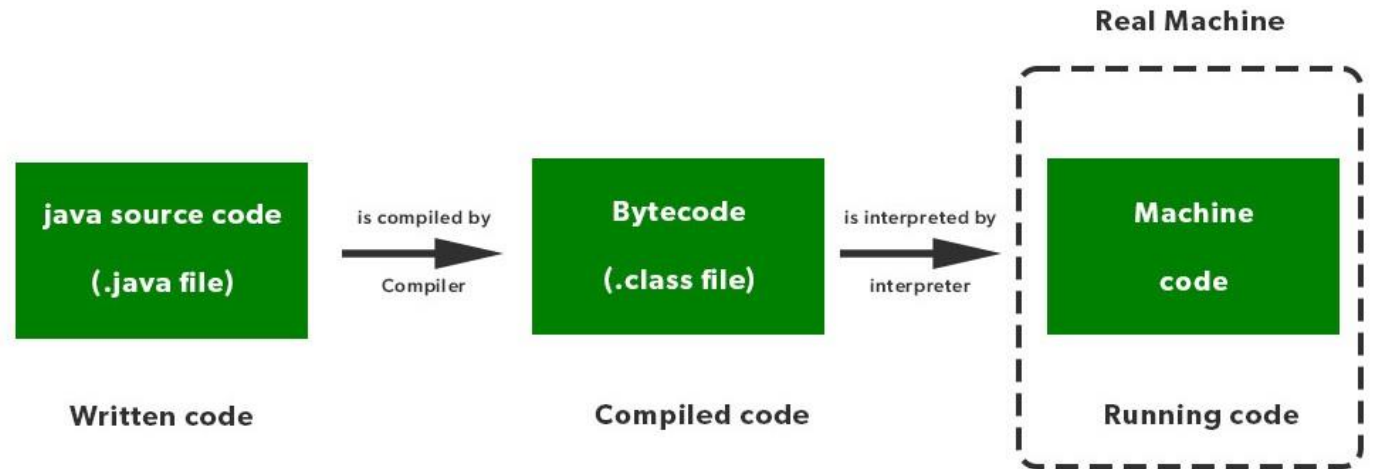
■ **Characteristics:**

- Translated by a **Compiler**
- Entire source code is analyzed and converted into a native executable
- Code is optimized during compilation
- No compiler required on the target system to run the program
- Source code is not directly visible

Onhexgroup.ir

Hybrid (Interpreted & Compiled) Programming Languages

- **Java**
- Source code is first compiled into **Bytecode**
- During execution, the **Java Virtual Machine (JVM)** translates bytecode into



Telegram: onhex_ir

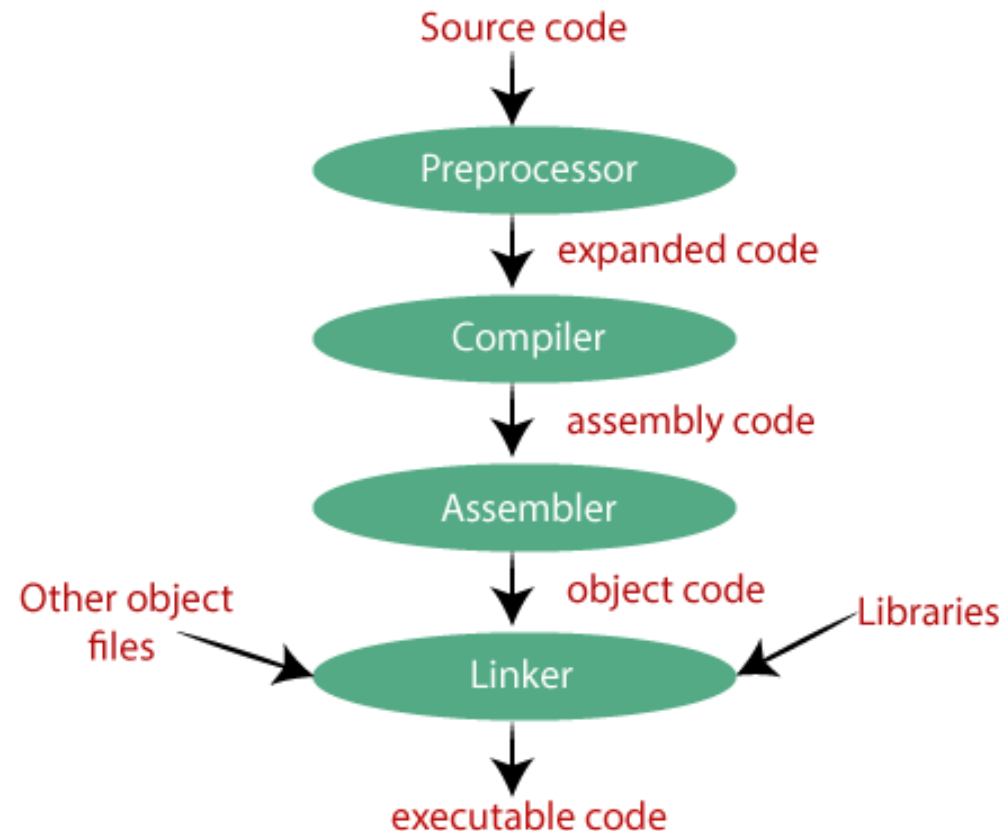
Course Focus

- **In this course, we cover:**
 - **x86 Architecture**
 - Both **32-bit** and **64-bit** environments
 - **Protected Mode** execution
 - Reverse engineering of compiled programs (C / C++)

Onhexgroup.ir

How Compilers Work

■ Stages of Converting C Source Code into an Executable:



Onhexgroup.ir

Preprocessor

- **Adds preprocessor-related information to the source code before compilation**
- **Preprocessor directives start with #**
 - **Examples: #include, #error, #define, etc.**
- **Input: .c file**
- **Output: .i file**
- **cl /P /EP code.c**

Telegram: onhex_ir

Compiler

- **Converts preprocessed code into assembly instructions**
- **Input: .i file**
- **Output: .asm file**
- **cl /c /Facode.asm /Tccode.i**

Onhexgroup.ir

Assembler

- **Converts assembly code into an object file**
- **Object files contain machine code (object code) but are not directly executable**
- **Input: .asm file**
- **Output: .obj file**
- **`ml.exe /c /coff /Zi /F1 /Fo code.obj code.asm`**

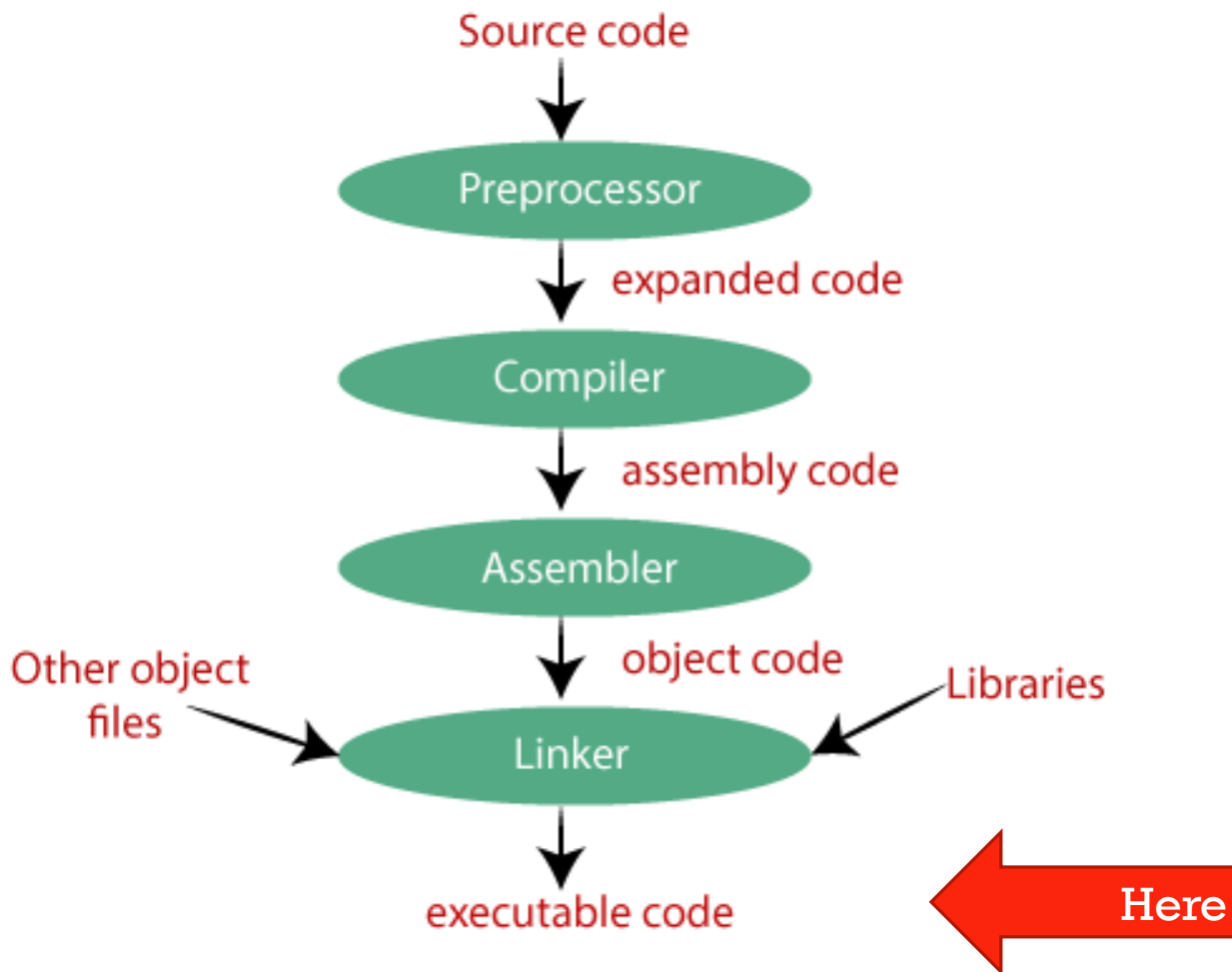
Onhexgroup.ir

Linker

- **Links all required object files and libraries**
- **Produces the final executable file**
- **Input: .obj file**
- **Output: .exe file**
- **link code.obj**

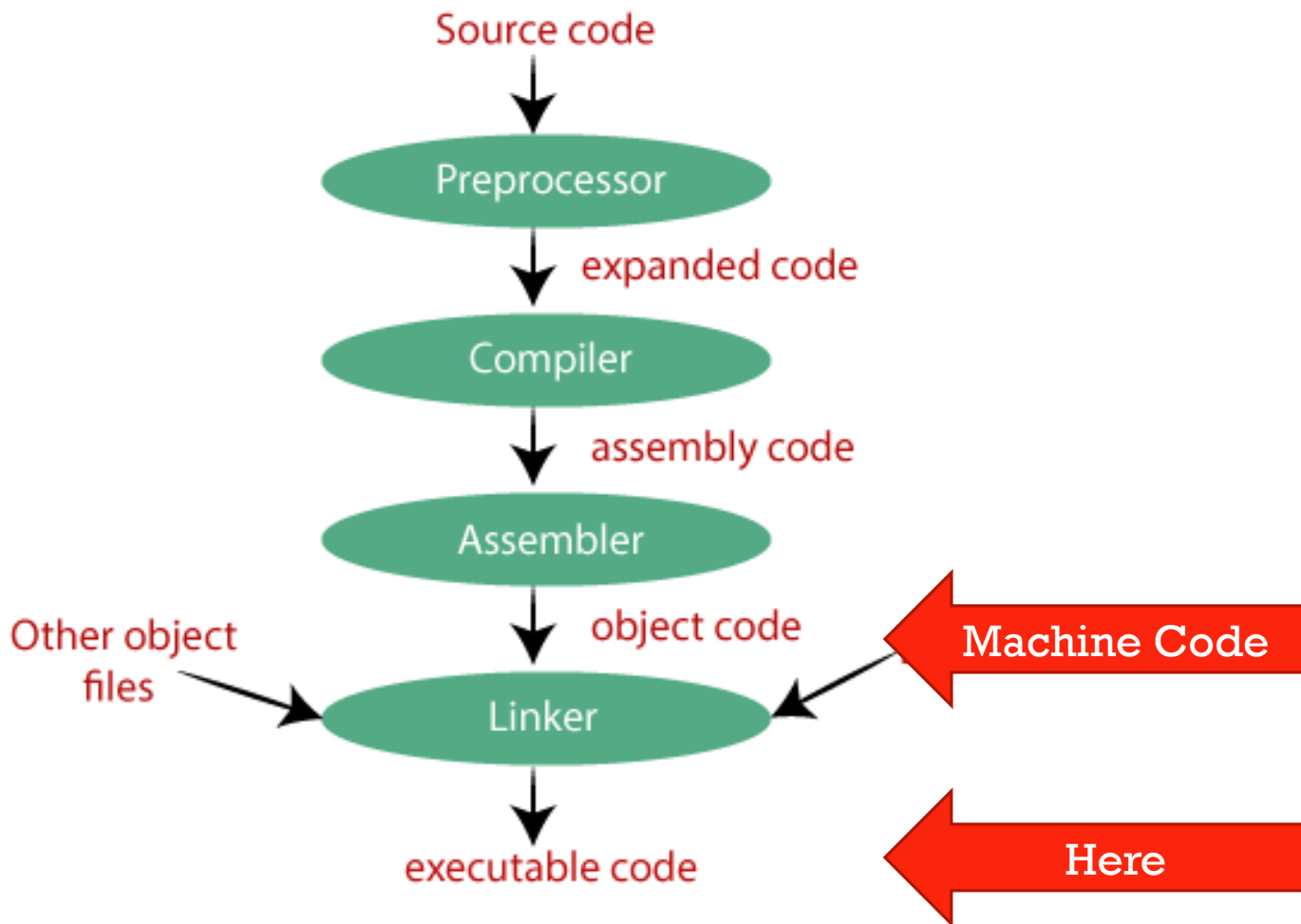
Onhexgroup.ir

Importance of Learning Assembly Language



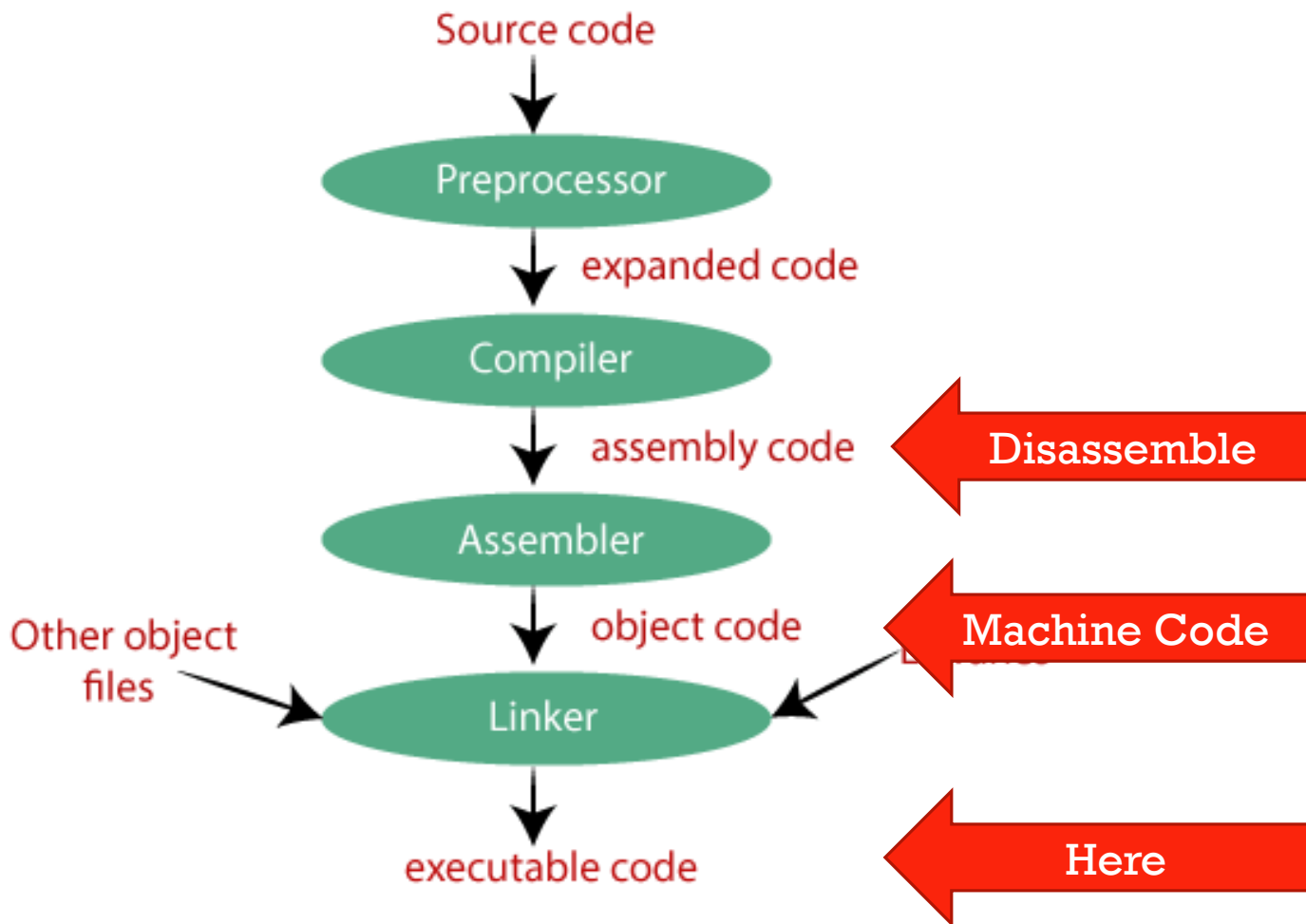
Onhexgroup.ir

Importance of Learning Assembly Language



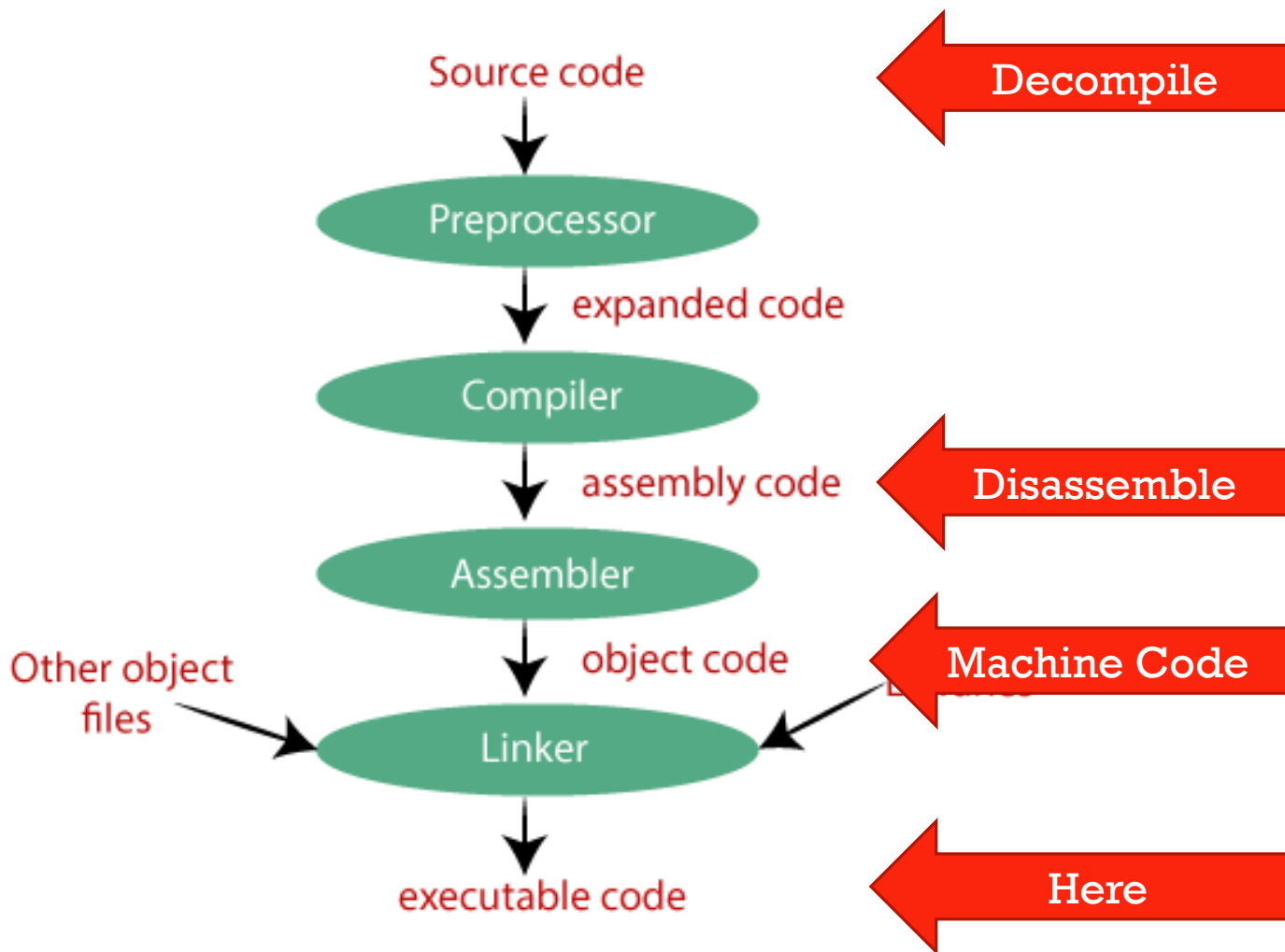
Onhexgroup.ir

Importance of Learning Assembly Language



Onhexgroup.ir

Importance of Learning Assembly Language



Onhexgroup.ir

References

- <https://learn.microsoft.com/en-us/cpp/build/walkthrough-compile-a-c-program-on-the-command-line?view=msvc-170>
- <https://www.studytonight.com/c/c-compilation-process.php>
- https://en.wikipedia.org/wiki/Object_file
- <https://www.chakray.com/programming-languages-types-and-features/>